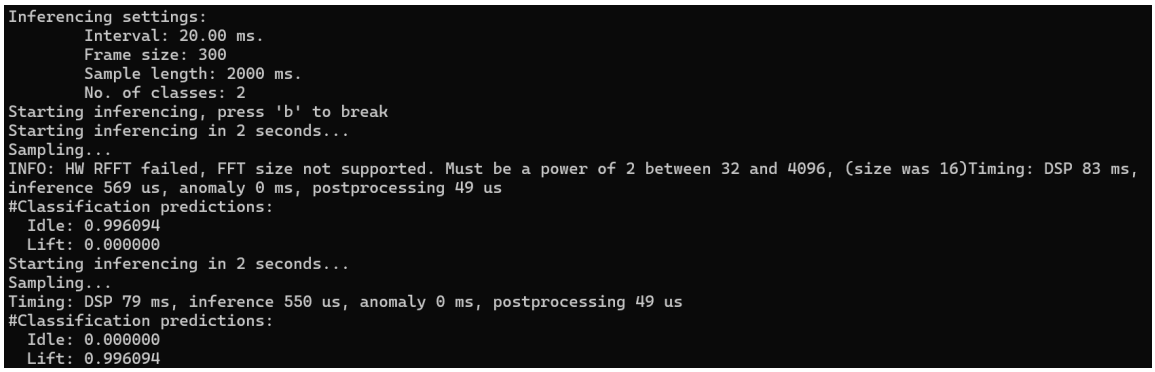
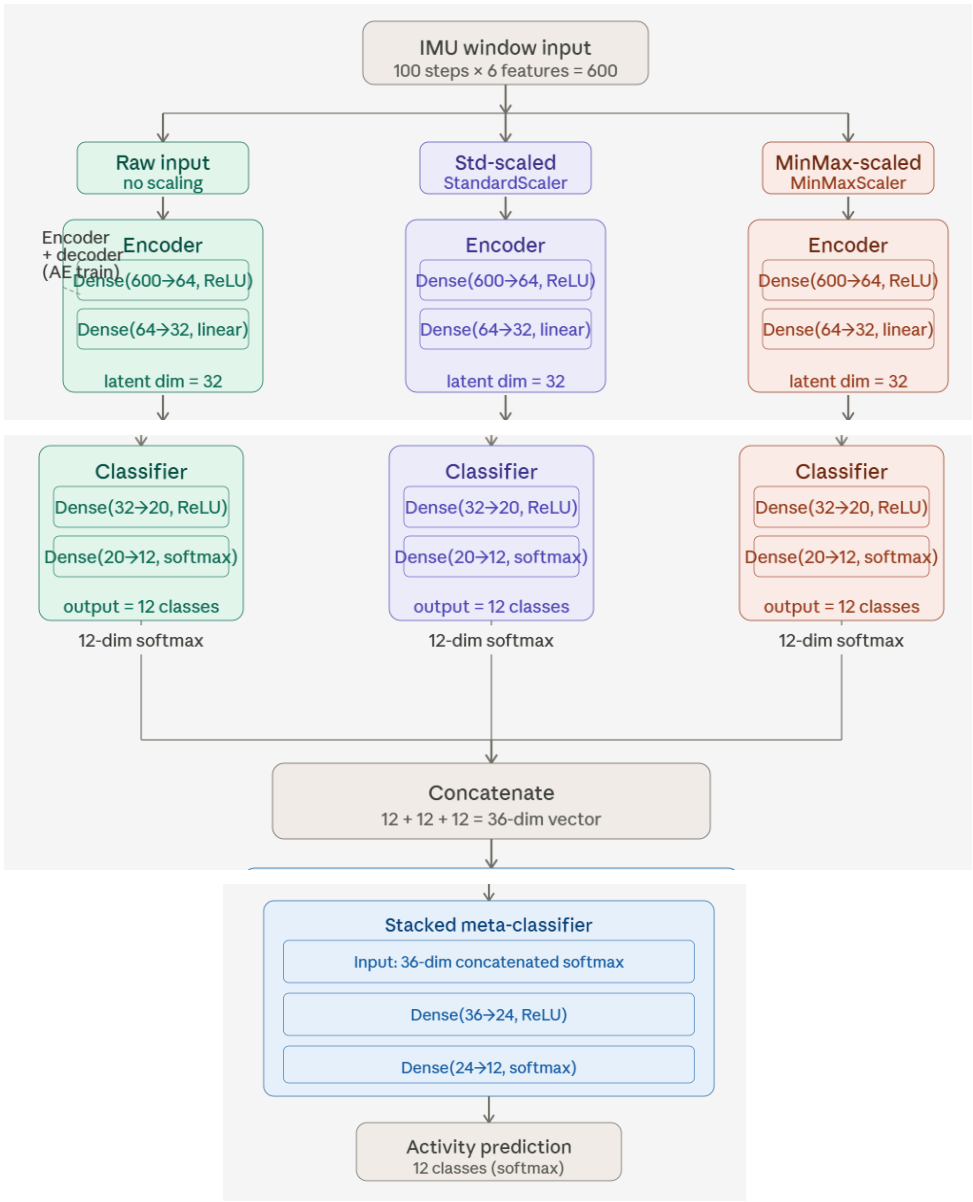
Task 1

- Link to public Edge Impulse project: https://studio.edgeimpulse.com/public/1010194/live
- Screenshot showing idle and lift state:



Task 2

Question 1



Question 2

- How pruning is applied and what sparsity schedule is used
    - Pruning is applied using TensorFlow Model Optimization's prune_low_magnitude function, which wraps each Dense layer in a PruneLowMagnitude wrapper. During training, the wrapper iteratively zeros out the smallest-magnitude weights according to a PolynomialDecay sparsity schedule. This schedule starts at 10% sparsity and ramps up to 80% sparsity by step 500, following a polynomial curve. The UpdatePruningStep callback is passed to model.fit so that the pruning masks are updated at each training step. The result is a model where 80% of the weights in each Dense layer are forced to zero, leaving only the 20% of weights with the largest magnitudes active.
- Why the pruning wrappers are stripped before QAT

    - After pruning, strip_pruning is called to remove the PruneLowMagnitude wrapper objects and return a standard Keras model. This is necessary because the pruning wrappers add auxiliary variables (step counters, masks, threshold tensors) that QAT does not know how to handle. The quantize_model API expects a clean Sequential or Functional model with only standard layer types. Stripping also reduces the model to its true sparse weight structure — the zeros are baked directly into the Dense layer kernels — so the downstream QAT and TFLite conversion steps operate on the actual compressed weights rather than the wrapper scaffolding.
- How QAT is applied after pruning
    - After stripping, each Dense layer is wrapped in a QuantizeWrapper using the Default8BitQuantizeRegistry, which inserts fake quantization nodes around the layer's weights and activations. During QAT fine-tuning, these fake quantization nodes simulate the rounding errors that will occur when the model is later converted to int8, allowing the weights and activations to adapt to quantization noise while still training in floating point. This produces a model whose learned parameters are robust to the precision loss of integer arithmetic.
- Why the notebook uses an additional mask-enforcement step during and after QAT instead of using only a simple pruning-then-QAT pipeline
    - A naive pruning-then-QAT pipeline has a critical flaw: QAT fine-tuning will gradually restore the previously pruned (zeroed) weights to nonzero values, undoing the sparsity that pruning achieved. This happens because the optimizer continues to compute gradients for all weights including the zeros, and the fake quantization noise creates small gradient signals that nudge zeroed weights away from zero. To prevent this, the notebook uses a MaskEnforcerCallback that re-applies the original pruning masks at the end of every training batch and every epoch. The masks are derived from the stripped model's weight structure — any weight that was zero after pruning is kept at zero throughout QAT — ensuring that the final model retains the full 80% sparsity even after fine-tuning.
- How the final model is converted to an int8 TFLite model and why representative data is needed
    - The QAT model is converted using tf.lite.TFLiteConverter. The converter is configured with Optimize.DEFAULT, TFLITE_BUILTINS_INT8 as the target ops, and tf.int8 for both input and output types, which forces full integer quantization. To convert floating-point scales and zero-points into integer equivalents, the converter needs to know the typical range of activations flowing through each layer. This is provided via a representative_dataset generator that feeds 200 samples from the training set through the model. Without this calibration data, the converter cannot determine the min/max activation ranges needed to compute accurate int8 scale factors, and would either fall back to float or produce poorly calibrated quantization ranges that hurt accuracy.
- How pruning and int8 quantization help when deploying to a resource-constrained device such as the Arduino Nano 33 BLE Sense

- The Arduino Nano 33 BLE Sense has extremely limited resources — 256 KB of flash for storing the model and 32 KB of SRAM for runtime buffers. Pruning and int8 quantization each attack this constraint from a different angle. Pruning zeros out 80% of the weights, which reduces the number of multiplications needed during inference and allows sparse computation to skip zero-weight operations, lowering both latency and power consumption. Int8 quantization reduces each weight from a 32-bit float (4 bytes) to an 8-bit integer (1 byte), shrinking the model size by approximately 4×. Together, these two techniques make it feasible to store and run a multi-layer neural network within the tight flash and RAM budget of a microcontroller that has no operating system, no GPU, and no dynamic memory allocation.

Question 3

Serial monitor output for resting board:

```
----------------------------------------
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.9961, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Standard-scaled model scores: 0.6211, 0.0273, 0.2656, 0.0156, 0.0469, 0.0000, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.0156
Min-max-scaled model scores: 0.0000, 0.0273, 0.0469, 0.0039, 0.1445, 0.0234, 0.5937, 0.1602, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.1055, 0.1016, 0.1328, 0.0664, 0.1055, 0.0508, 0.0859, 0.0937, 0.1250, 0.0430, 0.0625, 0.0312
----------------------------------------
Final prediction: Lying down
Class index: 2
Confidence score: 0.1328
----------------------------------------
```

Explanation: The output is acceptable. The model predicts "Lying down" for a stationary board resting flat on a surface, which is a reasonable label since the board is not moving. However, the confidence is relatively low at 0.1328, meaning the model is not strongly committed to any single class. In a well-calibrated deployment, a stationary board should produce a stable, high-confidence prediction. The low confidence suggests the model is uncertain, which is a meaningful signal that something in the deployment pipeline is not perfectly aligned.
One issue is that there is a scaling mismatch between training and deployment. The training data was drawn from the mHealth dataset, which was collected with specific sensor hardware calibrated to particular physical units. The Arduino Nano 33 BLE Sense IMU outputs raw accelerometer and gyroscope readings in its own units and range, and unless the Arduino sketch applies the exact same StandardScaler or MinMaxScaler parameters used during training (the fitted means, standard deviations, min, and max values), the input distribution seen at inference will differ from what the model was trained on. This scaling mismatch is a common cause of low confidence and unstable predictions. Another issue is that there may be an axis orientation mismatch. The mHealth dataset was collected with sensors attached to specific body locations and orientations, while the Arduino board placed flat on a desk has a different physical orientation. The accelerometer axes on the board may not correspond to the axes expected by the training data, causing the model to receive a rotated version of the expected gravity vector.

One final issue is sensor noise on the Arduino's IMU being continuous even when the board is stationary, and without any smoothing or filtering in the sketch, each 100-sample inference window will contain small fluctuations that shift predictions from frame to frame.
One practical improvement is to embed the fitted scaler parameters directly in the Arduino sketch. After training, the StandardScaler means and standard deviations (or the MinMaxScaler min and max values) should be exported as constant arrays and applied to the raw IMU readings before they are fed into the model. This ensures the input distribution at inference matches the distribution the model was trained on. Another improvement is to apply a simple majority-vote or exponential moving average over several consecutive inference windows before committing to a predicted label. Since individual windows can be noisy, aggregating predictions over a short buffer of frames (for example, 5–10 consecutive windows) smooths out transient fluctuations and produces more stable output on the serial monitor.

Question 4

Yes, I observed some unexpected behavior after deploying the ensemble model on the Arduino Nano 33 BLE Sense. Even when I shook the board aggressively, the model predicted "Sitting and relaxing" rather than "Jogging" or "Running." This kind motion-insensitive prediction indicates a systematic input mismatch rather than a model accuracy problem.
The most likely root cause is feature scaling mismatch. The three ensemble branches were each trained on data preprocessed with a fitted StandardScaler or MinMaxScaler whose parameters (means, standard deviations, min and max values) were derived from the mHealth dataset. If the Arduino sketch feeds raw IMU readings directly into the model without applying those same scaling transformations, every input window will land in a completely different region of the feature space than what the model learned from. The model then defaults to whatever class is most compatible with the out-of-distribution input, often a low-motion class like "Sitting and relaxing" because the unscaled gravity component of the accelerometer dominates the signal and looks superficially similar to a stationary posture in raw units.
A related issue is unit and range mismatch. The mHealth dataset accelerometer values are recorded in specific physical units (typically mg or m/s^2) with a particular dynamic range, while the Arduino Nano 33 BLE Sense IMU outputs values in its own native range depending on the configured full-scale setting. If the full-scale range of the Arduino IMU does not match the range of the training data, aggressive shaking may actually saturate or compress the signal into a range the model associates with gentle movement.
Axis orientation is another contributing factor. The mHealth dataset was collected with sensors mounted on specific body locations (chest, wrist, ankle) in defined orientations. The Arduino board placed flat on a surface has a different gravity vector decomposition across its three accelerometer axes compared to the sensor placements used during training. This means even a simple resting condition produces a different axis pattern, and dynamic motions like shaking produce patterns that do not resemble any training class clearly.
Finally, the motion patterns used during training are body-worn activity sequences (walking, jogging, climbing stairs) which have characteristic rhythmic frequency content tied to human gait. Manually shaking a bare circuit board produces a very different frequency signature and

amplitude envelope than any training class, so the model has no good match and gravitates toward whichever class has the most overlap with the noise floor of the unscaled input. Collectively, these factors mean the model is not actually evaluating the motion; it is pattern-matching against a distorted input that looks similar regardless of how the board moves, producing a consistently wrong but internally consistent prediction.